

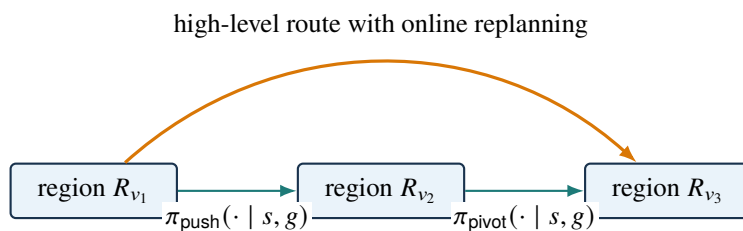
---

# A Concrete Research Plan for Hierarchical Reinforcement Learning and Graph Planning in Contact-Rich Manipulation

Technical memo for advisor–student discussion

July 24, 2026

**Central proposal.** Represent manipulation as a graph of robust state-space regions. Each directed graph edge is a set-to-set option with a prescribed contact template. The low-level controller is learned, while the high level performs risk-aware graph search and replans after every option. A small number of shared contact-template policies can instantiate many graph edges.



# Contents

|           |                                                                                      |           |
|-----------|--------------------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Executive summary</b>                                                             | <b>1</b>  |
| <b>2</b>  | <b>From the navigation graph to a manipulation option graph</b>                      | <b>1</b>  |
| 2.1       | The exact analogy . . . . .                                                          | 1         |
| 2.2       | Formal state and hybrid contact description . . . . .                                | 2         |
| 2.3       | Nodes, options, and graph edges . . . . .                                            | 3         |
| 2.4       | Why object pose alone is often not Markov . . . . .                                  | 3         |
| <b>3</b>  | <b>Concrete manipulation examples</b>                                                | <b>4</b>  |
| 3.1       | Example 1: planar fixture/contact maze – recommended first study . . . . .           | 4         |
| 3.1.1     | Physical layout . . . . .                                                            | 4         |
| 3.1.2     | Recommended numerical specification . . . . .                                        | 4         |
| 3.1.3     | Mode-specific rewards and guards . . . . .                                           | 5         |
| 3.2       | Example 2: extrinsic reorientation with a table edge . . . . .                       | 6         |
| 3.3       | Example 3: keyed or bent-channel insertion . . . . .                                 | 6         |
| 3.4       | Example 4: shelf or cubby extraction . . . . .                                       | 7         |
| 3.5       | Which example should be first? . . . . .                                             | 7         |
| <b>4</b>  | <b>Concrete algorithm: Mode-Constrained Option Graphs</b>                            | <b>7</b>  |
| 4.1       | Overview . . . . .                                                                   | 7         |
| 4.2       | Low-level goal-conditioned skill learning . . . . .                                  | 8         |
| 4.3       | Learning edge success and transition models . . . . .                                | 9         |
| 4.4       | Handoff compatibility . . . . .                                                      | 9         |
| 4.5       | Risk-aware graph search . . . . .                                                    | 10        |
| 4.6       | Optional extension: jointly optimize discrete edges and continuous portals . . . . . | 11        |
| <b>5</b>  | <b>Experimental protocol</b>                                                         | <b>11</b> |
| 5.1       | Research questions and predicted signatures . . . . .                                | 11        |
| 5.2       | Baselines required for a credible comparison . . . . .                               | 11        |
| 5.3       | Training-budget accounting . . . . .                                                 | 12        |
| 5.4       | Train/test splits . . . . .                                                          | 12        |
| 5.5       | Metrics . . . . .                                                                    | 13        |
| 5.6       | Ablation matrix . . . . .                                                            | 13        |
| <b>6</b>  | <b>Implementation details for the first study</b>                                    | <b>13</b> |
| 6.1       | Suggested software architecture . . . . .                                            | 13        |
| 6.2       | Initial hyperparameters . . . . .                                                    | 14        |
| 6.3       | Contact-template guards . . . . .                                                    | 14        |
| 6.4       | Graph construction and adaptive refinement . . . . .                                 | 15        |
| <b>7</b>  | <b>Failure modes and diagnostic responses</b>                                        | <b>16</b> |
| <b>8</b>  | <b>Recommended staged research plan</b>                                              | <b>16</b> |
| <b>9</b>  | <b>What would constitute convincing evidence?</b>                                    | <b>17</b> |
| <b>10</b> | <b>Positioning and related work</b>                                                  | <b>17</b> |
| <b>11</b> | <b>Immediate student deliverables</b>                                                | <b>18</b> |

# 1 Executive summary

The prior Dubins-car maze experiment has a clean decomposition: rooms are graph nodes, doorway transitions are graph edges, and each learned edge policy moves the vehicle from a region in one room to a target region in an adjacent room. The same idea can be transferred to contact-rich manipulation, but the abstraction must include enough contact and robot configuration information to make skill handoffs predictable.

The most important refinement is:

## Key design decision

A graph edge need not own a separate neural network. Train a small library of reusable, goal-conditioned policies indexed by a coarse contact template. Each graph edge is an *instantiation* of one policy with a source region, target region, contact specification, and continuous parameters. Four shared policies can implement hundreds of graph edges.

The recommended first task is a **planar contact maze**: a rectangular or T-shaped object is manipulated through a modular board of rooms, narrow portals, turning pockets, and recontact bays by two independently actuated planar fingertips. The graph nodes are object-pose regions augmented with canonical contact interfaces; edge templates are single-finger push, anchored pivot, pinch translation, and recontact. This task is the closest manipulation analogue of the nine-room navigation experiment and permits controlled scaling in path length, branching factor, contact switches, and dynamics uncertainty.

The main algorithm is a **Mode-Constrained Option Graph (MCOG)**:

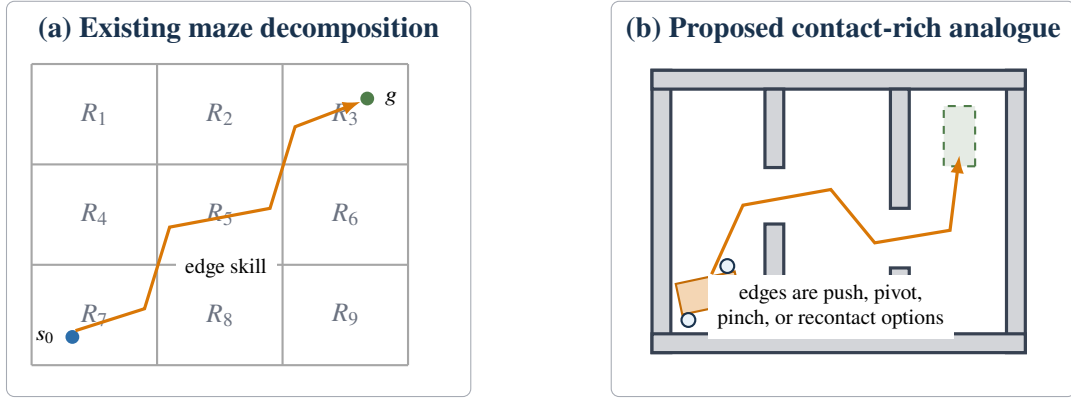
1. construct regions and candidate mode-labeled edges from geometry;
2. train goal-conditioned low-level options with SAC+HER, with PPO as an algorithm-independence replication;
3. estimate each edge’s success probability, duration, force cost, terminal-state distribution, and handoff compatibility;
4. plan with a risk-aware shortest-path or stochastic shortest-path objective; and
5. execute one option, observe the resulting state, and replan.

The experiments should isolate four claims: local learnability, compositional generalization, robustness through detection and replanning, and the value of a contact-aligned abstraction. The decisive baselines are not only flat SAC/PPO, but also flat policies given the same local-reset curriculum and abstract labels, a hierarchy built from an equally sized arbitrary partition, a skill library without explicit graph search, and a learned high-level policy over the same low-level options.

# 2 From the navigation graph to a manipulation option graph

## 2.1 The exact analogy

In the maze study, a room is not a point; it is a set of continuous states. Likewise, a manipulation node should be a robust set of object and robot states. A doorway policy does not aim for one exact vehicle state; it aims for a portal set from which the next policy can reliably begin. The corresponding manipulation concept is a *canonical interface set*: a set of object poses, velocities, contact configurations, and gripper poses that supports reliable continuation.



**Figure 1:** The transfer is set-to-set, not point-to-point. In both domains, a discrete route composes local continuous controllers. In manipulation, the abstract state must generally include contact and gripper-interface information in addition to object pose.

## 2.2 Formal state and hybrid contact description

Let the simulator or robot state be

$$s = (x_o, \dot{x}_o, q_r, \dot{q}_r, z_c), \quad s \in \mathcal{S}, \quad (1)$$

where  $x_o \in SE(2)$  or  $SE(3)$  is the object pose,  $q_r$  is the robot or fingertip configuration, and  $z_c$  contains measured or inferred contact information. For a candidate contact  $c$ , an exact contact mode can be represented as

$$m_c \in \{\text{separate}, \text{stick}, \text{slide}^+, \text{slide}^-\}. \quad (2)$$

For example, normal contact obeys the complementarity relation

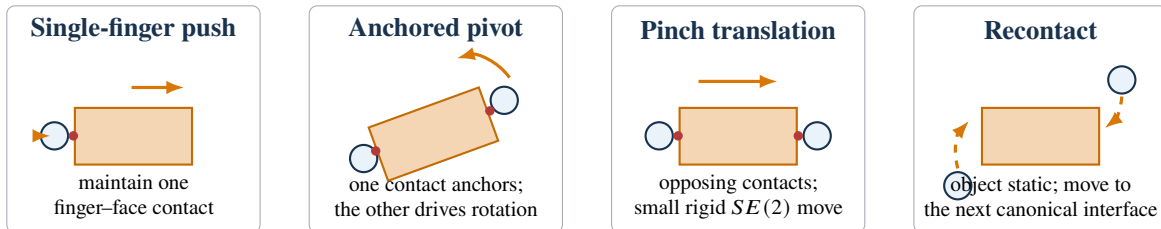
$$0 \leq \lambda_{n,c} \perp \psi_c(q_r, x_o) \geq 0, \quad (3)$$

where  $\psi_c$  is signed separation and  $\lambda_{n,c}$  is normal contact force or impulse. Sticking and sliding further constrain tangential relative velocity  $v_{t,c}$  and friction force  $\lambda_{t,c}$ .

Exact mode enumeration is usually too fine for a learning architecture. Instead define a small set of *contact templates*

$$\mathcal{K} = \{\text{push}, \text{pivot}, \text{pinch}, \text{recontact}\}, \quad (4)$$

where a template groups several exact stick-slip micro-modes that serve the same functional role. A template-specific guard  $\chi_k(s, a) \in \{0, 1\}$  checks required contacts, forbidden contacts, force limits, and gross slip-direction constraints.



**Figure 2:** The four recommended coarse contact templates. These are functional templates, not necessarily exact complementarity modes. A shared goal-conditioned policy is trained for each template.

### 2.3 Nodes, options, and graph edges

Let the abstract graph be  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ . Each node  $v \in \mathcal{V}$  owns a robust region

$$R_v \subset \mathcal{S}, \quad (5)$$

and the abstraction map  $\phi : \mathcal{S} \rightarrow \mathcal{V}$  returns the region containing the current state. A useful product-node form is

$$v = (C_i, \Theta_j, \Gamma_\ell), \quad (6)$$

where  $C_i$  is an object-position cell,  $\Theta_j$  is an orientation interval, and  $\Gamma_\ell$  is a canonical contact/gripper interface.

A directed edge is

$$e = (v, w, k, \xi_e), \quad e \in \mathcal{E}, \quad (7)$$

where  $k \in \mathcal{K}$  is a contact template and  $\xi_e$  contains edge-specific parameters such as active finger, object face, target portal, desired terminal orientation, and force limits. The associated option is

$$\omega_e = (I_e, \pi_{\theta_k}(a \mid s, g_e, \xi_e), \beta_e), \quad (8)$$

with initiation set  $I_e \subseteq R_v$ , shared template policy  $\pi_{\theta_k}$ , target set  $g_e \subseteq R_w$ , and termination rule  $\beta_e$ .

The distinction between *edge identity* and *network identity* is essential:

$$e_1 \neq e_2 \quad \text{can still use} \quad \pi_{\theta_{k(e_1)}} = \pi_{\theta_{k(e_2)}}. \quad (9)$$

For example, one shared push network can instantiate every “push through east portal” and “push through north portal” edge after transforming observations into object-centric coordinates.

### 2.4 Why object pose alone is often not Markov

Two states with the same object pose can have different outgoing reachability because the robot may be on different sides of the object, a contact may be preloaded or slipping, or the gripper may be near a kinematic limit. There are two clean remedies:

1. **Canonical interfaces:** require each option to terminate at low velocity in a standardized contact/gripper set  $\Gamma_\ell$ .
2. **Product graph:** explicitly augment the node with contact class, grasp family, approach side, or a small robot-configuration bin.

The first experiment should use canonical interfaces because it exposes failures cleanly. A later experiment can relax canonicalization and learn state-dependent transition kernels.

An empirical abstraction-aliasing score is

$$A(v, e) = \text{Var}_{s \sim \mu_v} [\hat{p}_e(s)] + \lambda \mathbb{E}_{s, s' \sim \mu_v} W_1(\hat{K}_e(\cdot \mid s), \hat{K}_e(\cdot \mid s')), \quad (10)$$

where  $\hat{p}_e$  is edge success probability and  $\hat{K}_e$  is the terminal-state distribution. Large  $A(v, e)$  means the node is too coarse and should be split.

### 3 Concrete manipulation examples

#### 3.1 Example 1: planar fixture/contact maze – recommended first study

##### 3.1.1 Physical layout

Use a top-down planar board assembled from reusable modules:

- open rooms in which the object can translate or rotate;
- narrow portals that constrain orientation;
- one-sided corridors that favor pushing;
- turning pockets that require an anchored pivot;
- recontact bays that provide room to switch finger configurations;
- at least two routes between some start and goal pairs.

A rectangular object creates simple face contacts; a T-shaped object makes the global geometry more interesting while keeping local contacts interpretable. Use two independently actuated circular fingertips in the initial scientific prototype. This removes arm kinematic singularities and makes the contact-mode question itself the main difficulty.

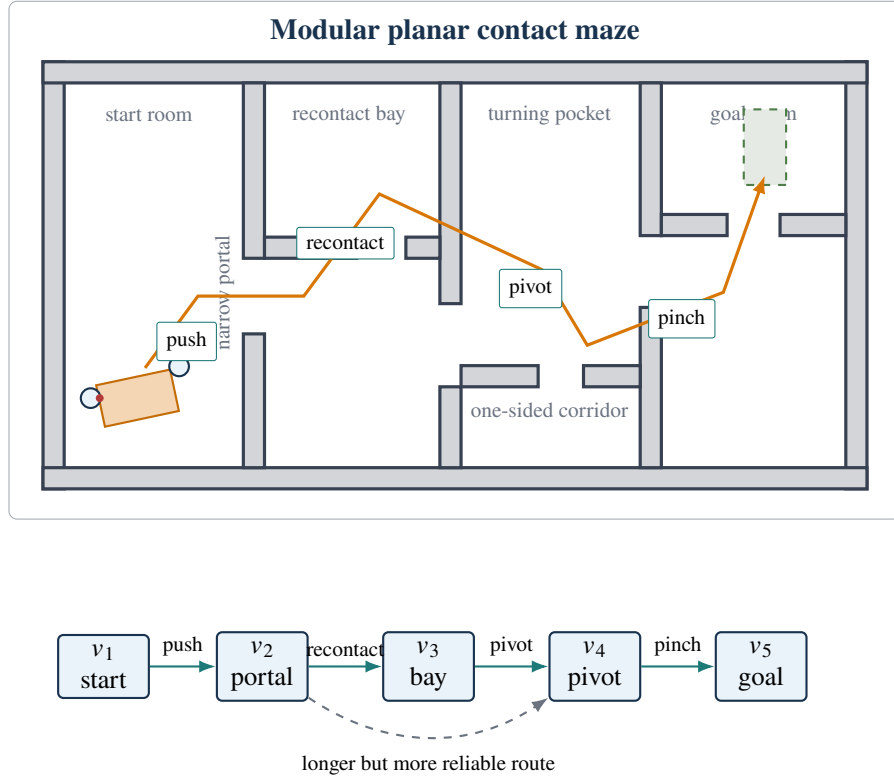
##### Important caveat

A standard parallel-jaw gripper does not provide independent planar motion of the two fingertips. The proposed “one finger anchors while the other slides” behavior is therefore not a primitive degree of freedom on a Panda hand. For the clean first study, use independent planar fingertips. For a later Panda experiment, redefine templates around wrist motion, gripper opening/closing, and environment-assisted pivots.

##### 3.1.2 Recommended numerical specification

##### Concrete starting specification

|                   |                                                                                                                      |
|-------------------|----------------------------------------------------------------------------------------------------------------------|
| Board             | 0.80 m × 0.60 m planar workspace; modular walls and removable gates.                                                 |
| Object            | Rectangle 0.10 m × 0.06 m, then a T-shape of similar footprint; mass randomized in [0.15, 0.30] kg.                  |
| Fingertips        | Two independently actuated discs of radius 0.012 m; planar position control through velocity or impedance commands.  |
| Training friction | Object–table coefficient $\mu \sim U[0.20, 0.60]$ ; fingertip friction $\mu_f \sim U[0.5, 1.0]$ .                    |
| Portal clearance  | Object width plus 5–30 mm, divided into easy, medium, and hard levels.                                               |
| Control           | Physics at 500 Hz, policy at 20–50 Hz; option horizon $T_o = 100$ –250 control steps.                                |
| Episode           | Up to 10 options or 1500 control steps; replanning after every option and after guard failure.                       |
| Observation       | Object pose/velocity, fingertip poses/velocities, contact impulses or binary contacts, target portal description.    |
| Action            | $a = (v_x^L, v_y^L, v_x^R, v_y^R)$ with an impedance inner loop; optional normal-force setpoints in a later version. |



**Figure 3:** A concrete planar benchmark and one corresponding abstract route. The board should include alternative routes so that edge reliability, not only edge count, affects the high-level plan.

A concrete graph node is

$$v = (C_i, \Theta_j, \Gamma_\ell), \quad (11)$$

with  $C_i$  one board cell or portal neighborhood,  $\Theta_j$  one of eight orientation bins, and  $\Gamma_\ell$  one of a small set of canonical interfaces. To avoid a full Cartesian product, instantiate only geometrically reachable combinations. A portal node may use a tighter position/orientation set than an open-room node.

Example initiation and target sets for an eastward single-finger push are

$$I_e = \{s : x_o \in C_i, \theta_o \in \Theta_j, d(f_L, \text{west face}) \leq \epsilon_c, \|\dot{x}_o\| \leq \epsilon_v\}, \quad (12)$$

$$G_e = \{s : x_o \in P_{i \rightarrow r}, \theta_o \in \Theta_{j'}, \|\dot{x}_o\| \leq \epsilon_v, \Gamma(s) = \Gamma_{\text{west-push}}\}, \quad (13)$$

where  $P_{i \rightarrow r}$  is the portal interface between cells.

### 3.1.3 Mode-specific rewards and guards

For edge  $e$ , use a shaped local reward

$$r_e(s, a, s') = R_g \mathbb{I}[s' \in G_e] - w_d d_e(s') - w_a \|a\|_2^2 - w_F [F(s') - F_{\max}]_+^2 - w_m (1 - \chi_{k(e)}(s', a)) - w_T, \quad (14)$$

where  $d_e$  measures object-centric distance to the target interface. The guard terminates and labels an option failure when a required contact is lost for more than  $n_{\text{grace}}$  steps, a forbidden contact appears, the object leaves the board, or forces exceed a safety threshold.

A useful curriculum samples initial states from progressively larger subsets:

$$I_e^{(1)} \subset I_e^{(2)} \subset \dots \subset I_e, \quad (15)$$

starting near the portal and expanding backward into the source region. The flat-policy baseline must receive an equivalent reset curriculum in a control experiment.

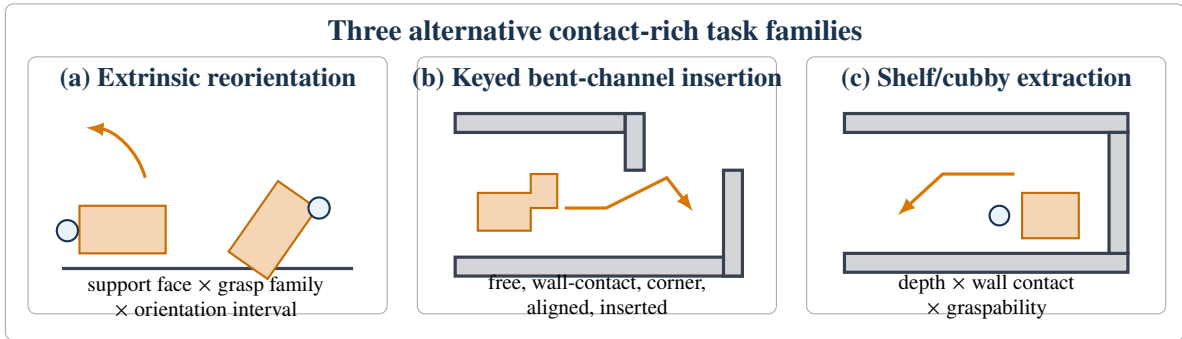
### 3.2 Example 2: extrinsic reorientation with a table edge

The goal is to bring a cuboid to an arbitrary target orientation and an acceptable final grasp using finger contacts and the table or fixture as an external contact. This is a strong second task because orientation reachability is naturally discrete at the level of support faces but continuous within each support/grasp family.

Define nodes as

$$v = (f_{\text{support}}, g_{\text{grasp}}, \Theta_j), \quad (16)$$

where  $f_{\text{support}}$  is the object face on the table,  $g_{\text{grasp}}$  is a pair of contacted faces and contact-location bin, and  $\Theta_j$  is a local orientation interval. Candidate edges are table-edge pivot, finger-anchored pivot, slide-in-grasp, roll to an adjacent support face, and regrasp.



**Figure 4:** Three follow-on environments. Each has a natural discrete contact or support structure, but all retain continuous within-mode control.

A concrete evaluation query samples a target support face and target grasp family, not only a target orientation. This prevents the policy from ending in an orientation that is correct but unusable for a downstream task.

### 3.3 Example 3: keyed or bent-channel insertion

A standard peg-in-hole task often has a nearly fixed high-level sequence and is therefore a weak test of graph planning. A better benchmark uses a T-shaped or keyed object and a channel with two entrances, a bend, asymmetric clearances, and guide walls. Nodes can be

$$v = (z_{\text{progress}}, c_{\text{env}}, \Theta_j), \quad (17)$$

where  $c_{\text{env}} \in \{\text{free, left-wall, right-wall, corner, two-wall, inserted}\}$ . Edges are compliant approach, wall-follow, corner pivot, centering, insertion, and retract/recover.

The graph should contain at least two strategic routes: follow the left guide wall or the right guide wall. Friction and clearance randomization can make one route shorter but less reliable, creating a meaningful risk-aware planning problem.

### 3.4 Example 4: shelf or cubby extraction

Place an object deep in a shelf where direct grasping is impossible. The robot must push or pivot the object, exploit wall contact, pull it into a graspable zone, and then grasp. Nodes can include depth interval, object orientation interval, active shelf contact, and a binary graspability label. This task is hardware-relevant and naturally supports learned preconditions, but perception and occlusion introduce additional confounds; it should follow the planar studies rather than precede them.

### 3.5 Which example should be first?

**Table 1:** Recommended ordering of environments. Scores are qualitative and intended to guide project staging.

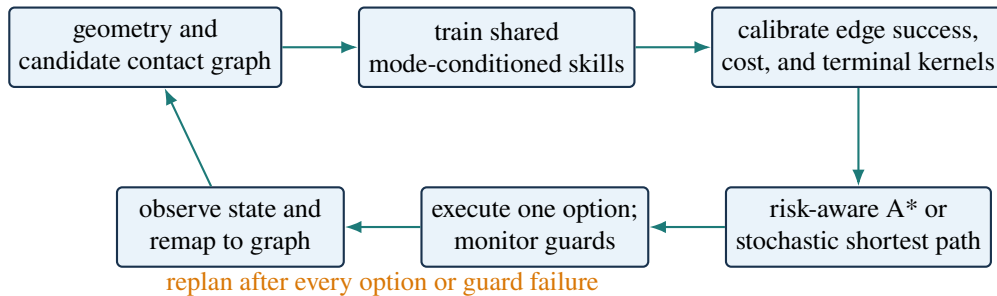
| Task                    | Maze analogy | Contact structure | Clean diagnosis | Hardware relevance |
|-------------------------|--------------|-------------------|-----------------|--------------------|
| Planar contact maze     | Excellent    | High              | Excellent       | Medium             |
| Extrinsic reorientation | Good         | Excellent         | Good            | High               |
| Bent-channel insertion  | Good         | Excellent         | Good            | High               |
| Shelf extraction        | Moderate     | High              | Moderate        | Excellent          |

## 4 Concrete algorithm: Mode-Constrained Option Graphs

### 4.1 Overview

The proposed algorithm has four learned components and one explicit planner:

1. contact-template policies  $\{\pi_{\theta_k}\}_{k \in \mathcal{K}}$ ;
2. edge success models  $\hat{p}_e(s)$ ;
3. edge cost models for duration, energy, force, and switching;
4. terminal-state kernels  $\hat{K}_e(s_T | s_0)$  or their abstract transition probabilities; and
5. a risk-aware high-level graph search or stochastic shortest-path solver.



**Figure 5:** Offline learning and online closed-loop planning. The graph is not executed open loop.

## 4.2 Low-level goal-conditioned skill learning

For template  $k$ , train a policy

$$\pi_{\theta_k}(a \mid o(s), \rho(g), \xi), \quad (18)$$

where  $o(s)$  is an object-centric observation,  $\rho(g)$  encodes the target region or portal, and  $\xi$  encodes contact-side and edge parameters. Object-centric coordinates and symmetry augmentation allow the same network to share data across board locations and left–right mirror cases.

For SAC, optimize the maximum-entropy objective

$$J_k(\pi) = \mathbb{E}_{\tau \sim \pi_{\theta_k}} \left[ \sum_{t=0}^{T_o-1} \gamma^t (r_k(s_t, a_t, s_{t+1}) + \alpha \mathcal{H}(\pi_{\theta_k}(\cdot \mid s_t, g))) \right]. \quad (19)$$

A standard twin-critic target is

$$y_t = r_t + \gamma(1 - d_t) \left[ \min_{i \in \{1,2\}} Q_{\tilde{\phi}_i}(s_{t+1}, a', g) - \alpha \log \pi_{\theta_k}(a' \mid s_{t+1}, g) \right]. \quad (20)$$

Use Hindsight Experience Replay by relabeling failed local rollouts with achieved terminal portal states or intermediate achieved poses [4, 3].

For the PPO replication, use the clipped objective

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[ \min \left( r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad r_t(\theta) = \frac{\pi_{\theta}(a_t \mid s_t, g)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t, g)}. \quad (21)$$

PPO is useful as an algorithm-independence check, but SAC+HER is the primary recommendation because the local task is goal-conditioned and off-policy replay is valuable [2].

---

### Algorithm 1 Training a shared contact-template policy

---

**Input:** Template  $k$ , local task set  $\mathcal{Q}_k$ , curriculum  $\{I^{(j)}\}$

**Output:** Policy  $\pi_{\theta_k}$  and replay buffer  $\mathcal{D}_k$

- 1: Initialize actor, twin critics, target critics, and replay buffer
  - 2: **for**  $n = 1$  to interaction budget  $B_k$  **do**
  - 3:   Sample an edge instance  $e \sim \mathcal{Q}_k$  and curriculum level  $j$
  - 4:   Reset  $s_0 \sim I_e^{(j)}$ ; sample target set  $g_e \subset R_w(e)$
  - 5:   **for**  $t = 0$  to  $T_o - 1$  **do**
  - 6:     Sample  $a_t \sim \pi_{\theta_k}(\cdot \mid s_t, g_e, \xi_e)$  and step simulator
  - 7:     Compute reward in Equation (14) and guard  $\chi_k$
  - 8:     Store  $(s_t, a_t, r_t, s_{t+1}, g_e, d_t)$  in  $\mathcal{D}_k$
  - 9:     **if** goal reached, guard fails, or horizon reached **then**
  - 10:       **break**
  - 11:   Add HER relabels using achieved object poses and portal sets
  - 12:   Update critics, actor, and entropy coefficient from minibatches
  - 13:   Advance curriculum when held-out local success exceeds threshold
-

### 4.3 Learning edge success and transition models

For each instantiated edge  $e$ , collect held-out rollouts from states spanning  $I_e$ . Estimate

$$\hat{p}_e(s) \approx \mathbb{P}(s_T \in G_e \mid s_0 = s, \omega_e), \quad (22)$$

$$\hat{\tau}_e(s) \approx \mathbb{E}[T_e \mid s_0 = s], \quad (23)$$

$$\hat{f}_e(s) \approx \mathbb{E} \left[ \sum_t [F_t - F_{\text{safe}}]_+^2 \mid s_0 = s \right], \quad (24)$$

$$\hat{K}_e(s_T \mid s) \approx p(s_T \mid s_0 = s, \omega_e). \quad (25)$$

For a first version, use a small MLP for  $\hat{p}_e(s)$  and a categorical distribution over terminal abstract nodes for  $\hat{K}_e$ . Calibrate  $\hat{p}_e$  with temperature scaling or isotonic regression on a held-out validation set.

A simple conservative success estimate uses a Beta posterior. If edge  $e$  has  $n_e^+$  successes and  $n_e^-$  failures under a uniform prior,

$$p_e \mid \mathcal{D}_e \sim \text{Beta}(1 + n_e^+, 1 + n_e^-), \quad (26)$$

and a lower confidence estimate is

$$\underline{p}_e = \text{BetaInv}(\delta; 1 + n_e^+, 1 + n_e^-). \quad (27)$$

State-dependent models can use an ensemble and replace  $\underline{p}_e$  by a lower ensemble quantile.

The abstract transition model is

$$\hat{P}(v' \mid v, e) = \frac{N(v, e, v') + \eta}{N(v, e) + \eta|\mathcal{V}|}, \quad (28)$$

where unexpected but recoverable outcomes map to other graph nodes rather than an absorbing failure state.

---

#### Algorithm 2 Calibrating instantiated graph edges

---

**Input:** Graph candidates  $\mathcal{E}$ , policies  $\{\pi_{\theta_k}\}$ , trials per edge  $M$

**Output:** Edge models  $\hat{p}_e, \hat{\tau}_e, \hat{f}_e, \hat{P}(\cdot \mid v, e)$

- 1: **for all**  $e = (v, w, k, \xi_e) \in \mathcal{E}$  **do**
  - 2:     **for**  $m = 1$  to  $M$  **do**
  - 3:         Sample  $s_0 \sim I_e$  using a space-filling design
  - 4:         Roll out  $\omega_e$  with guard monitoring
  - 5:         Record success, duration, force cost, terminal state, terminal node
  - 6:     Fit and calibrate  $\hat{p}_e(s)$ ; estimate cost models and transition counts
  - 7:     Compute conservative probability  $\underline{p}_e(s)$
  - 8:     **if**  $\mathbb{E}_{s \sim I_e}[\underline{p}_e(s)] < p_{\min}$  **then**
  - 9:         prune or mark edge high-cost
- 

### 4.4 Handoff compatibility

A chain can fail even when every option succeeds locally because the predecessor's terminal distribution may lie outside the successor's training distribution. For adjacent edges  $e$  and  $e'$ , define

$$H(e, e') = \mathbb{E}_{s_T \sim \hat{K}_e} [\hat{p}_{e'}(s_T)]. \quad (29)$$

A low handoff score indicates that the node hides relevant continuous state. Remedies are:

1. tighten or canonicalize the predecessor target set;

2. enlarge the successor initiation curriculum;
3. split the shared node by contact or robot-interface class; or
4. fine-tune the predecessor with a successor-aware terminal reward

$$r_{\text{handoff}}(s_T) = \lambda_H \log \left( \epsilon + \max_{e' \in \text{Out}(w)} \hat{p}_{e'}(s_T) \right). \quad (30)$$

Terminal-state compatibility is known to be a central issue in skill chaining [7].

If each edge succeeds independently with probability  $p$ , a length- $L$  open-loop chain succeeds with probability  $p^L$ . For  $p = 0.98$  and  $L = 10$ , this is only  $0.98^{10} \approx 0.817$ . Replanning and recovery are therefore part of the scientific method, not optional implementation polish.

## 4.5 Risk-aware graph search

For deterministic abstract outcomes, assign edge cost

$$c_e(s) = w_\tau \hat{\tau}_e(s) + w_f \hat{f}_e(s) + w_a \hat{u}_e(s) - w_p \log \left( \max\{\underline{p}_e(s), \epsilon\} \right) + w_s \mathbb{I}[k(e) \neq k(e_{\text{prev}})], \quad (31)$$

where  $\hat{u}_e$  is control effort and the last term penalizes contact-template switches. Minimizing the sum of negative log probabilities approximately maximizes path reliability because

$$\arg \max_P \prod_{e \in P} p_e = \arg \min_P \sum_{e \in P} -\log p_e. \quad (32)$$

Use Dijkstra or A\* when each option has one dominant terminal node. A simple admissible heuristic is a lower bound on the number of remaining portals times the minimum edge cost.

When outcomes are stochastic, solve a stochastic shortest-path problem:

$$V(v) = \min_{e \in \text{Out}(v)} \left[ c(v, e) + \sum_{v'} \hat{P}(v' | v, e) V(v') \right], \quad V(v_{\text{goal}}) = 0. \quad (33)$$

If handoff cost depends on edge pairs, plan on the line graph whose states are  $(v, e_{\text{prev}})$  and add  $-\lambda_H \log H(e_{\text{prev}}, e)$ .

---

### Algorithm 3 Online planning, execution, and recovery

---

**Input:** Current state  $s$ , goal set  $G$ , graph and calibrated edge models

**Output:** Success or declared unrecoverable failure

- 1: **while**  $s \notin G$  and option budget remains **do**
  - 2:   Map  $v \leftarrow \phi(s)$ ; remove edges whose state-dependent precondition is false
  - 3:   Compute risk-aware route  $P = (e_1, \dots, e_L)$  using A\* or Equation (33)
  - 4:   **if** no route exists **then**
  - 5:     invoke recovery node or return failure
  - 6:   Execute only  $e_1$  with policy  $\pi_{\theta_{k(e_1)}}$
  - 7:   Monitor contact guard, force, and progress; abort early on violation
  - 8:   Observe terminal state  $s \leftarrow s_T$  and update online edge statistics
  - 9:   Replan from the observed node, including unexpected recoverable outcomes
  - 10: **return** success iff  $s \in G$
-

## 4.6 Optional extension: jointly optimize discrete edges and continuous portals

The first system should use a small finite set of target portals per edge. A later extension can optimize continuous subgoals  $g_e$  jointly with the discrete route:

$$\min_{(e_1, \dots, e_L), g_{1:L}} \sum_{\ell=1}^L c_{e_\ell}(s_\ell, g_\ell) \quad \text{s.t.} \quad g_\ell \in G_{e_\ell}, \quad s_{\ell+1} \sim \hat{K}_{e_\ell}(\cdot \mid s_\ell, g_\ell). \quad (34)$$

Practical solvers can alternate graph search over edge sequences with gradient or sampling-based optimization of portal targets, or use learned edge value functions as local continuous optimizers.

## 5 Experimental protocol

### 5.1 Research questions and predicted signatures

**Table 2:** Falsifiable questions and the evidence that would support them.

| ID | Question                                                        | Supporting empirical signature                                                                                                                                     |
|----|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| H1 | Does decomposition improve aggregate sample efficiency?         | Hierarchy reaches fixed global success thresholds with fewer total environment transitions, after counting all low-level, edge-model, and high-level data.         |
| H2 | Does the hierarchy enable composition rather than memorization? | Skills trained on short routes and local modules transfer to longer paths and unseen global board assemblies without low-level fine-tuning.                        |
| H3 | Does explicit structure improve robustness?                     | Under perturbations, the hierarchy detects failures, replans, and recovers more often; its advantage grows with disturbance rate and path length.                  |
| H4 | Is contact-aligned structure important?                         | Contact-aligned regions outperform an equally sized geometric/random partition with the same policy capacity and training budget.                                  |
| H5 | Are interfaces the limiting factor?                             | Global success versus path length is predicted by edge success and handoff scores; terminal regularization improves long chains more than single-edge performance. |

### 5.2 Baselines required for a credible comparison

1. **Flat universal goal-conditioned SAC.** One policy receives the full state and final object goal.
2. **Flat universal goal-conditioned PPO.** Replicates the student’s existing navigation baseline.
3. **Flat + local reset curriculum.** Same resets and training-state coverage as the option policies, but no temporal hierarchy.
4. **Flat + abstract labels.** Receives cell, orientation-bin, and contact-template indicators, controlling for privileged representation.
5. **Shared option library without graph search.** A learned high-level policy selects  $(k, g)$  every option period.

6. **Generic continuous-subgoal HRL.** A HIRO-style high level proposes continuous subgoals to a lower-level controller [5].
7. **Arbitrary-partition hierarchy.** Same number of nodes, edges, and policies, but regions are geometric or random rather than contact-aligned.
8. **Oracle edge model.** Uses true empirical success and transition statistics; separates planning limitations from model-estimation limitations.
9. **Nominal model-based planner.** When available for planar pushing, provides a trajectory-quality reference rather than a directly fair learning baseline.

The option framework is the natural formal basis for this comparison [1]. Methods such as SEADS provide a relevant learned-symbolic-skill baseline when a known abstraction is available [6]; learned preconditions for contact-rich hybrid controllers provide a closely related search-based manipulation baseline [8].

### 5.3 Training-budget accounting

For every method, report total environment interactions

$$N_{\text{total}} = N_{\text{low}} + N_{\text{edge-model}} + N_{\text{high}} + N_{\text{fine-tune}}, \quad (35)$$

not only the largest component. Parallel simulation changes wall-clock time but not sample complexity. Also report network parameter count, number of hand-specified regions and templates, reset privileges, and any oracle contact labels.

A practical main-study budget schedule is

$$N \in \{0.25, 0.5, 1, 2, 4, 8, 16\} \text{ million aggregate transitions}, \quad (36)$$

with evaluation at each checkpoint. The hierarchy’s per-template budgets must sum to the same aggregate budget as the flat baseline. For a pilot, use five seeds; for the main planar experiment, use ten seeds and paired evaluation scenarios.

### 5.4 Train/test splits

Use non-overlapping evaluation axes:

1. **Held-out queries:** unseen start and goal states on training boards.
2. **Longer compositions:** train low-level policies on local options and tune the planner on paths of at most four edges; test on shortest paths of six to ten edges.
3. **Held-out topology:** assemble unseen boards from familiar room, corridor, pivot, and portal modules.
4. **Held-out geometry:** change portal width, wall placement, object aspect ratio, and turning-pocket size.
5. **Dynamics shift:** friction, mass, restitution, contact stiffness, actuator delay, and control-rate changes outside the central training distribution.
6. **Observation shift:** object-pose noise, contact-label noise, delayed force sensing, and partial loss of tactile information.
7. **Intervention tests:** impulse the object, displace a fingertip, or break contact midway through an option.

A suggested friction split is training  $\mu \in [0.20, 0.60]$ , interpolation test  $\mu \in [0.25, 0.55]$ , and extrapolation test  $\mu \in [0.10, 0.20) \cup (0.60, 0.80]$ .

## 5.5 Metrics

Primary sample-efficiency metrics are

$$N_\rho = \min\{N : \widehat{S}(N) \geq \rho\}, \quad (37)$$

$$\text{AULC} = \int_0^{N_{\max}} \widehat{S}(N) dN, \quad (38)$$

where  $\widehat{S}(N)$  is global task success after  $N$  aggregate interactions and  $\rho \in \{0.6, 0.8, 0.9\}$ .

Robustness metrics should include success under each shift, worst-decile success across randomized conditions, force-limit violations, object drops or escapes, and the fraction of disturbances recovered by replanning. A risk-sensitive summary is

$$R_{\text{CVaR}} = \text{CVaR}_{0.1} (\text{episode return over randomized physical parameters}). \quad (39)$$

Structural diagnostics should include:

- per-edge success and calibration error;
- predicted versus empirical path reliability;
- handoff score  $H(e, e')$ ;
- abstraction-aliasing score  $A(v, e)$  from Equation (10);
- success stratified by shortest abstract path length and number of contact switches;
- physical trajectory length, option count, and replanning count.

The central statistical test should include a **method  $\times$  path-length interaction**. The hypothesis predicts that the hierarchical advantage grows as more local behaviors must be composed.

## 5.6 Ablation matrix

# 6 Implementation details for the first study

## 6.1 Suggested software architecture

A custom environment is preferable to using a standard single-step task unchanged. ManiSkill provides useful infrastructure and simple primitives such as PushCube, PokeCube, PullCube, RollBall, and PushT, but the central benchmark should force multiple contact-template switches and offer alternative routes. The environment wrapper should expose:

- exact state and contact labels for the initial controlled study;
- a mode guard and failure reason;
- local edge reset samplers;
- target-set descriptions rather than only point goals;

**Table 3:** Ablations that distinguish why a hierarchy works or fails.

| Ablation                                                        | Scientific question                                                        |
|-----------------------------------------------------------------|----------------------------------------------------------------------------|
| Object pose only vs. pose + contact interface                   | Is contact information required for an approximately Markov abstraction?   |
| Canonical interfaces vs. unrestricted terminal states           | Does standardized handoff reduce long-chain failure?                       |
| Exact contact modes vs. coarse functional templates             | Is precise hybrid enumeration useful or unnecessarily brittle?             |
| Independent network per edge vs. shared network per template    | How much specialization is worth the policy-count and data cost?           |
| Contact-aligned vs. arbitrary partition                         | Does physical structure, rather than hierarchy alone, provide the benefit? |
| Deterministic edge cost vs. calibrated success/transition model | Is risk-aware planning important under stochastic contact outcomes?        |
| Open-loop route execution vs. option-level replanning           | How much robustness comes from closed-loop abstraction?                    |
| Oracle contact labels vs. estimated labels                      | How dependent is performance on privileged mode information?               |
| Cartesian velocity vs. impedance/force-velocity action layer    | Does a contact-aware control interface reduce learning burden?             |

- graph-node membership and portal membership; and
- logging of force, slip, option boundaries, and abstract outcomes.

A clean code separation is:

| Module                            | Responsibility                                             | Learned? |
|-----------------------------------|------------------------------------------------------------|----------|
| <code>geometry.py</code>          | cells, portals, feasible edge instances                    | no       |
| <code>contact_templates.py</code> | guards, parameterization, target-set tests                 | no       |
| <code>skill_trainer.py</code>     | SAC/PPO + HER per shared template                          | yes      |
| <code>edge_model.py</code>        | success, costs, calibration, abstract transitions          | yes      |
| <code>planner.py</code>           | $A^*$ , stochastic shortest path, line-graph handoff costs | no       |
| <code>executor.py</code>          | option rollout, guard monitoring, replanning               | mixed    |

## 6.2 Initial hyperparameters

These values are starting points, not claims of optimality.

Use separate critics per contact template initially. A universal actor with a template one-hot can be added as a sharing ablation. Normalize observations in object-centric coordinates and mirror left/right samples when the geometry is symmetric.

## 6.3 Contact-template guards

For a push option requiring left fingertip contact with object face  $f$  and no right-finger contact, an example guard is

$$\chi_{\text{push}}(s, a) = \mathbb{I}[d(L, f) \leq \epsilon_c] \mathbb{I}[\lambda_{n,L} \geq \lambda_{\min}] \mathbb{I}[d(R, \partial O) \geq \epsilon_{\text{sep}}] \mathbb{I}[F_{\max}(s) \leq F_{\text{abort}}]. \quad (40)$$

**Table 4:** Suggested initial low-level training settings.

| Quantity             | SAC + HER                                      | PPO replication                                     |
|----------------------|------------------------------------------------|-----------------------------------------------------|
| Actor/critic network | 3 hidden layers, 256 units, ReLU; twin critics | 3 hidden layers, 256 units, Tanh or ReLU            |
| Learning rate        | $3 \times 10^{-4}$ actor and critics           | $3 \times 10^{-4}$ with linear decay                |
| Discount             | $\gamma = 0.99$                                | $\gamma = 0.99$                                     |
| Target update        | $\tau = 0.005$                                 | n/a                                                 |
| Replay / rollout     | replay $10^6$ , batch 512                      | 2048–8192 steps/update across vector envs           |
| Entropy              | automatic temperature tuning                   | entropy bonus 0.0–0.01                              |
| HER                  | future strategy, 4 relabels per transition     | not standard; use goal-resampled on-policy rollouts |
| PPO clip / GAE       | n/a                                            | $\epsilon = 0.2$ , $\lambda_{\text{GAE}} = 0.95$    |
| Option horizon       | 100–250 policy steps                           | same                                                |
| Evaluation           | 200 held-out local starts per edge family      | same                                                |

For pinch, require two opposing contacts and a minimum inward normal-force sum. For pivot, require the anchor contact to remain within a contact-location tolerance while object angular progress has the intended sign. For recontact, require object speed below  $\epsilon_v$  and all object contacts absent until the final acquisition phase.

Guards should allow a short grace interval to avoid terminating on one noisy contact sample. Log guard violations separately from target misses so failure causes are interpretable.

## 6.4 Graph construction and adaptive refinement

The initial graph should be hand-constructed from known geometry. Candidate edges are generated by local feasibility tests:

1. source and target regions share a portal or a manipulation module;
2. the chosen contact template has a nonempty initiation set;
3. simple geometric reachability and collision checks pass; and
4. a short random or nominal rollout achieves nonzero success.

After collecting edge data, split a node if  $A(v, e)$  in Equation (10) is high for any important outgoing edge. A practical split procedure trains a shallow decision tree to predict edge success from candidate features  $(\theta_o, \Gamma, q_r)$  within the node; a high-gain split suggests a missing abstract variable. Merge neighboring nodes if their outgoing edge models and terminal kernels are statistically indistinguishable.

## 7 Failure modes and diagnostic responses

**Table 5:** Likely failure modes and the experiment or fix that diagnoses them.

| Observed failure                                          | Likely cause                                               | Diagnostic or response                                                              |
|-----------------------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------|
| High local success, low long-chain success                | terminal/initiation mismatch                               | measure $H(e, e')$ ; canonicalize interfaces or add successor-aware terminal reward |
| Different outcomes from visually identical node states    | abstraction aliases contact or robot configuration         | compute $A(v, e)$ ; split node by interface, orientation, or approach side          |
| Policy count grows exponentially                          | one-network-per-exact-mode design                          | share networks by coarse functional template; instantiate many edges per network    |
| Planner repeatedly chooses unreliable shortcuts           | success estimates are miscalibrated                        | reliability diagrams, Brier score, conservative lower bounds, online updates        |
| Hierarchy wins only with special resets                   | curriculum advantage, not execution hierarchy              | compare against flat policy with identical reset distribution                       |
| Mode violations dominate failures                         | action space or guard too weak                             | impedance/HFVC layer; stricter initiation sets; supervised warm starts              |
| Flat policy wins on short routes but loses on long routes | expected temporal-abstraction tradeoff                     | report method $\times$ path-length interaction rather than only mean success        |
| No unseen-topology transfer                               | local policies encode global coordinates or board identity | object-centric observations, symmetry augmentation, module-held-out testing         |

## 8 Recommended staged research plan

### Stage 0: reproduce and instrument the navigation result

Before changing domains, compute the same diagnostics proposed here on the Dubins maze: edge success, handoff compatibility, predicted path reliability, success versus graph distance, and arbitrary-partition baselines. This creates a methodological bridge and may reveal whether the original gain came from temporal abstraction, reset curriculum, or graph structure.

### Stage 1: planar independent-fingertip contact maze

Implement the rectangular-object board with push and recontact first. Add pivot, then pinch. Use oracle state/contact labels and canonical interface sets. Compare flat SAC/PPO, flat + curriculum, and the explicit graph hierarchy. The first publishable-quality figure should show global success versus both aggregate transitions and shortest abstract path length.

### Stage 2: stochastic edge models and perturbation recovery

Randomize friction and mass, inject contact breaks, learn calibrated edge success and transition models, and compare open-loop route execution with option-level replanning. This stage directly tests the robustness hypothesis.

### Stage 3: unseen board composition

Train all low-level policies on isolated modules or a subset of boards. Test on novel boards assembled from known modules and on longer paths. This is the clearest compositional-generalization result.

### Stage 4: embodied arm and environmental contacts

Move to a Panda-like arm or another physical robot. Redefine contact templates to match the available actuation: fingertip push, bilateral grasp, table-edge pivot, wall slide, and free-space regrasp. Use impedance or hybrid force–velocity control beneath RL. Extrinsic reorientation or shelf extraction is an appropriate target.

#### Falsifiable hypothesis

The project should be considered successful scientifically even if the hierarchy does not dominate everywhere. A particularly informative outcome would be a phase diagram showing that flat policies are preferable for short, dense-reward tasks, while graph-composed options become advantageous as path length, query diversity, and disturbance rate increase.

## 9 What would constitute convincing evidence?

The strongest result would satisfy all of the following:

1. The hierarchy reaches target global success rates with fewer *aggregate* transitions than flat baselines.
2. The advantage survives controls for local resets, abstract labels, and total network capacity.
3. The advantage grows with abstract path length or number of start–goal queries.
4. Low-level skills transfer without fine-tuning to unseen global graph compositions.
5. Contact-aligned partitions outperform equally sized arbitrary partitions.
6. Online replanning recovers a substantial fraction of injected disturbances.
7. Calibrated edge and handoff models predict route reliability well enough to change route selection rationally.

The most important negative result would also be interpretable: if individual edges are reliable but global success collapses with path length, then interface compatibility and abstraction quality, rather than local contact control, are the central scientific bottlenecks.

## 10 Positioning and related work

The proposed study sits at the intersection of temporal abstraction, goal-conditioned RL, learned skills for planning, and model-based contact planning. Options formalize temporally extended actions [1]; SAC, PPO, and HER provide the proposed low-level learning tools [3, 2, 4]; HIRO provides a generic learned-subgoal comparison [5]; SEADS learns skills as symbolic actions under a known abstraction [6]; terminal-state regularization addresses skill-chain compatibility [7]; and learned preconditions have been integrated with search for contact-rich manipulation in shelves [8].

Model-based work has also represented fixed contact modes and collision-free regions in graph-structured optimization. The distinction here should be explicit: the proposed contribution is not merely a contact graph, but a controlled

empirical study of *learned stochastic set-to-set options*, calibrated edge/handoff models, risk-aware replanning, and the resulting sample-efficiency and robustness tradeoffs. Graph-of-convex-sets contact planning is a useful nominal or oracle comparison where its assumptions apply [9].

## 11 Immediate student deliverables

A concrete two-week planning package should contain:

1. a one-page environment specification with exact state, action, object geometry, board modules, and contact-template guards;
2. a generated graph for at least two board layouts, including node and edge counts;
3. reset samplers and target-set tests for push and recontact edges;
4. a local SAC+HER baseline with held-out edge success maps over each initiation region;
5. a flat universal SAC baseline using exactly the same observations and aggregate interactions;
6. plots of edge success calibration, terminal distributions, and handoff score; and
7. a preregistered experiment table listing hypotheses, baselines, budgets, seeds, and test splits.

The first decision to lock is the embodiment. For a clean scientific comparison, the recommendation is two independently actuated planar fingertips. The first algorithmic milestone is not a learned high-level composer; it is a reliable explicit graph planner over learned push and recontact options with calibrated edge success and online replanning.

## References

- [1] R. S. Sutton, D. Precup, and S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. [https://doi.org/10.1016/S0004-3702\(99\)00052-1](https://doi.org/10.1016/S0004-3702(99)00052-1).
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017. <https://arxiv.org/abs/1707.06347>.
- [3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft Actor-Critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of ICML*, PMLR 80:1861–1870, 2018. <https://proceedings.mlr.press/v80/haarnoja18b.html>.
- [4] M. Andrychowicz, F. Wolski, A. Ray, et al. Hindsight Experience Replay. In *Advances in Neural Information Processing Systems*, 2017. <https://proceedings.neurips.cc/paper/2017/hash/453fadbd8a1a3af50a9df4df899537b5-Abstract.html>.
- [5] O. Nachum, S. Gu, H. Lee, and S. Levine. Data-efficient hierarchical reinforcement learning. In *Advances in Neural Information Processing Systems*, 2018. <https://papers.nips.cc/paper/7591-data-efficient-hierarchical-reinforcement-learning>.
- [6] J. Achterhold, M. Krimmel, and J. Stueckler. Learning temporally extended skills in continuous domains as symbolic actions for planning. In *Proceedings of CoRL*, PMLR 205:225–236, 2023. <https://proceedings.mlr.press/v205/achterhold23a.html>.

- [7] Y. Lee, J. J. Lim, A. Anandkumar, and Y. Zhu. Adversarial skill chaining for long-horizon robot manipulation via terminal state regularization. In *Proceedings of CoRL*, PMLR 164:406–416, 2022. <https://proceedings.mlr.press/v164/lee22a.html>.
- [8] J. Liang, X. Cheng, and O. Kroemer. Learning preconditions of hybrid force–velocity controllers for contact-rich manipulation. In *Proceedings of CoRL*, PMLR 205:679–689, 2023. <https://proceedings.mlr.press/v205/liang23a.html>.
- [9] B. P. Graesdal, S. Y. C. Chia, T. Marcucci, S. Morozov, A. Amice, P. A. Parrilo, and R. Tedrake. Towards tight convex relaxations for contact-rich manipulation. arXiv:2402.10312, 2024. <https://arxiv.org/abs/2402.10312>.